

Course Introduction

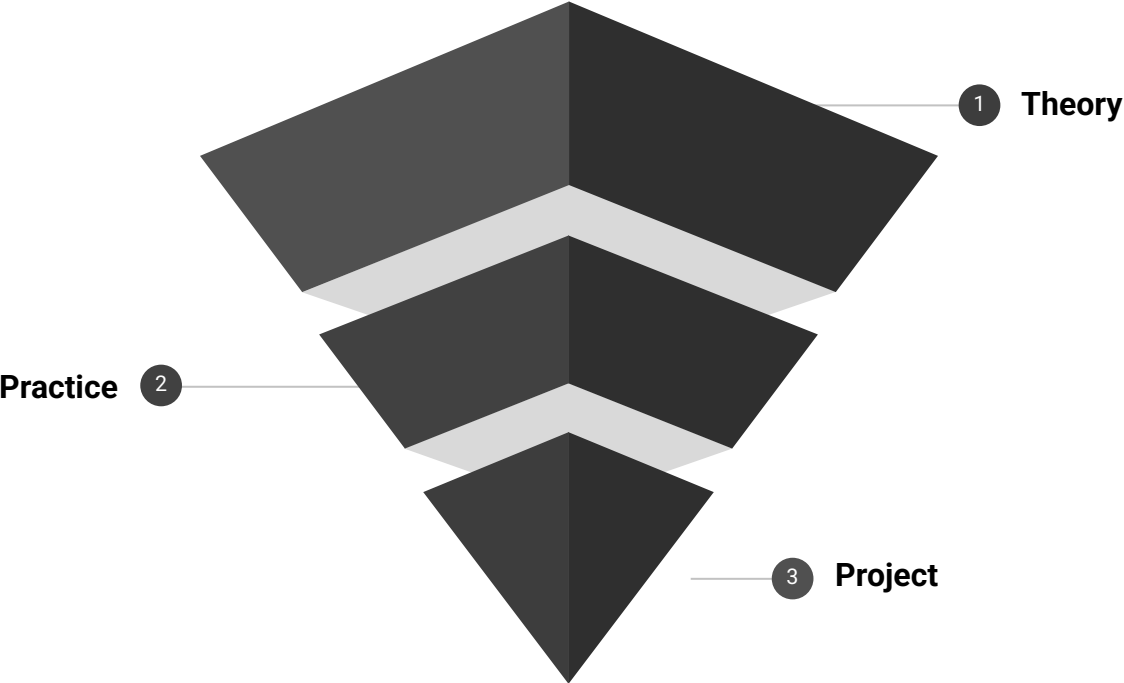


OBJECTIVES

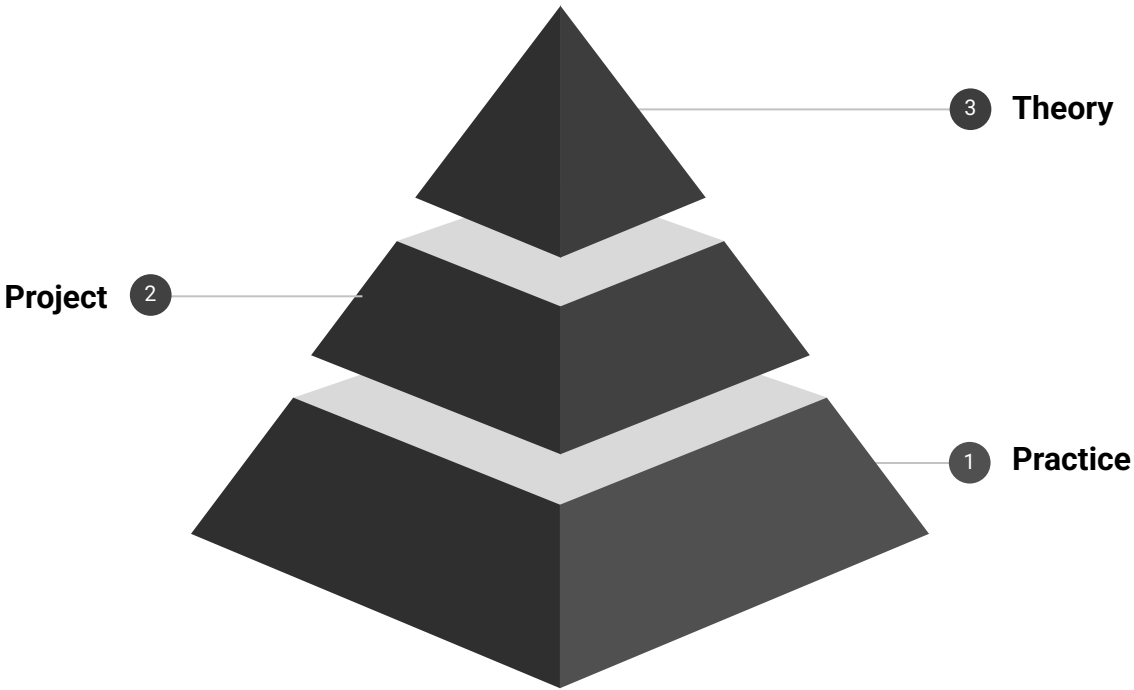
- Data-maturity model
- dbt and data architectures
- Data warehouses, data lakes, and lakehouses
- ETL and ELT procedures
- dbt fundamentals
- Analytics Engineering



TOP-DOWN



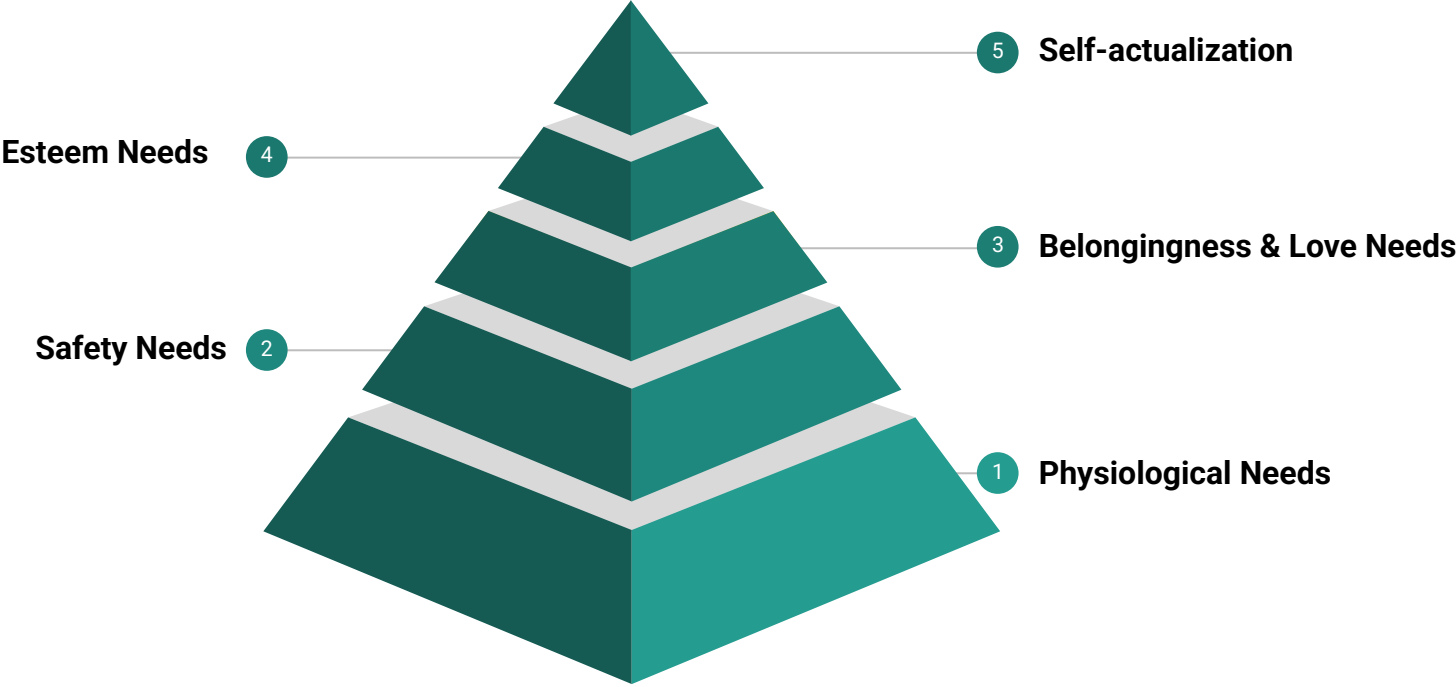
BOTTOM-UP



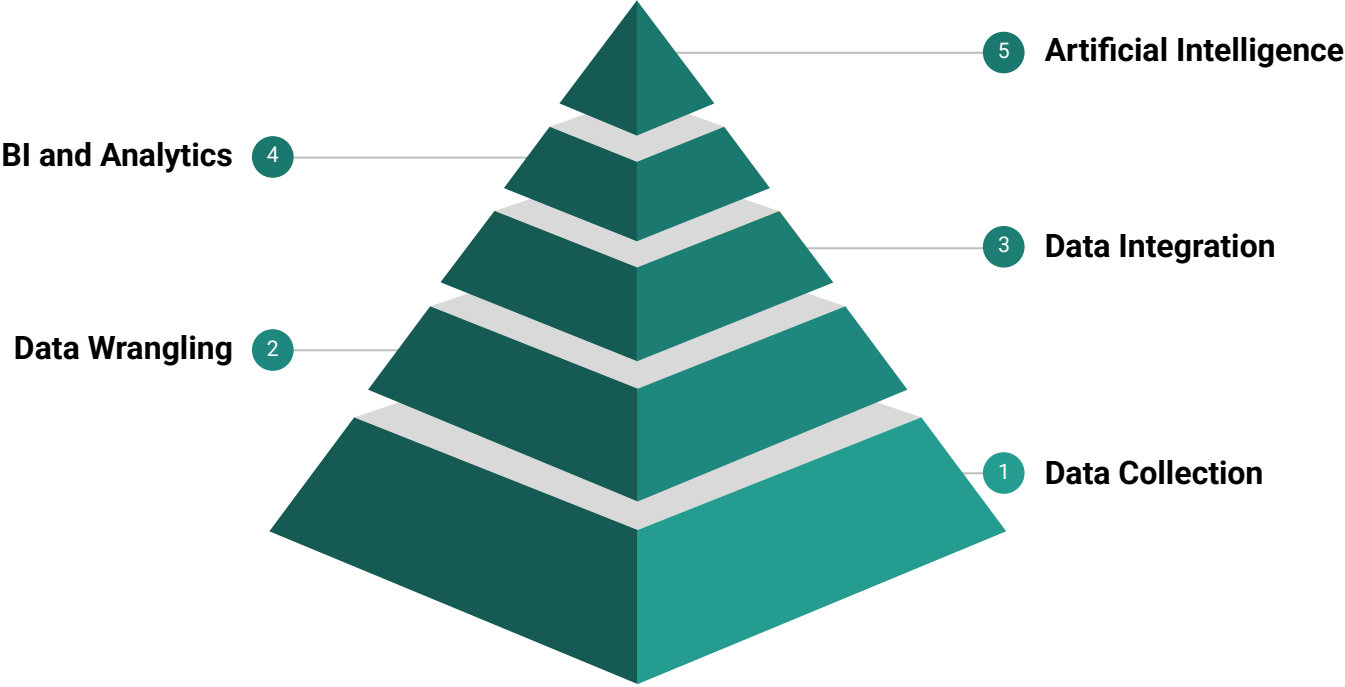
Data Maturity Model



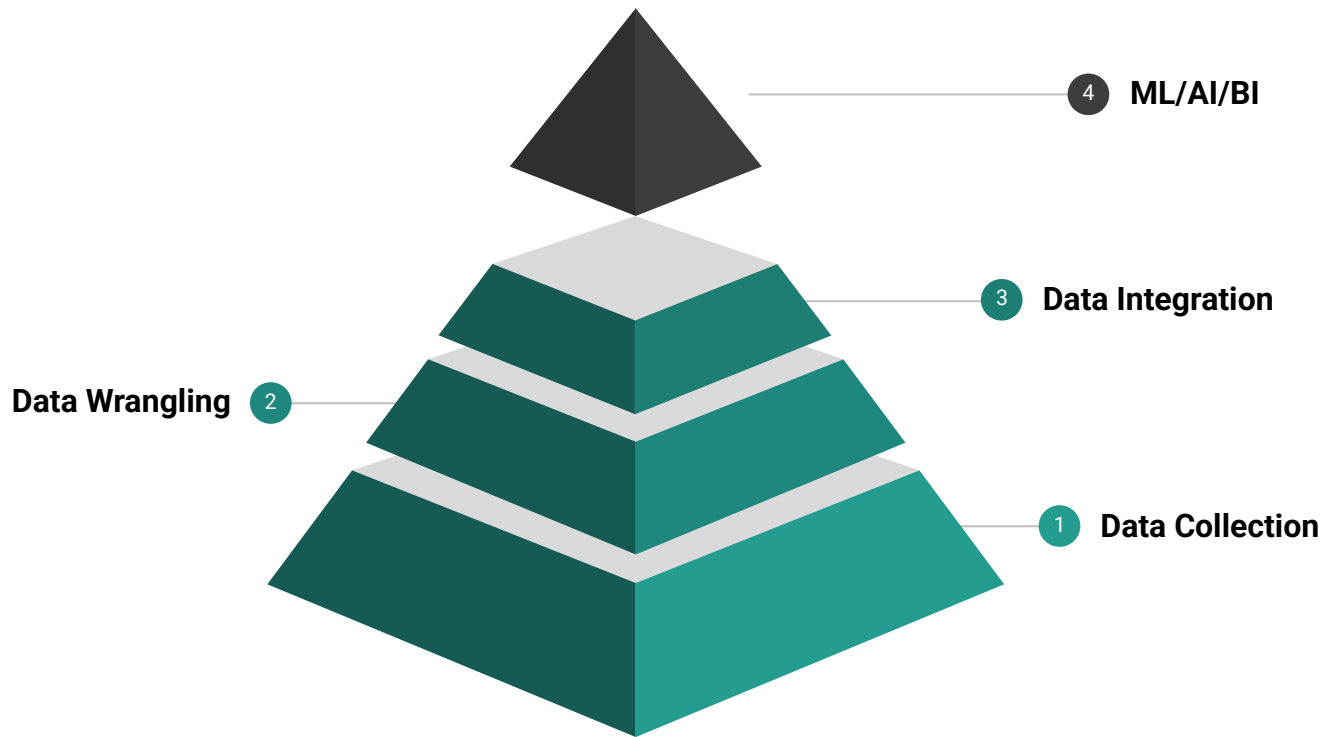
Maslow's Hierarchy of Needs



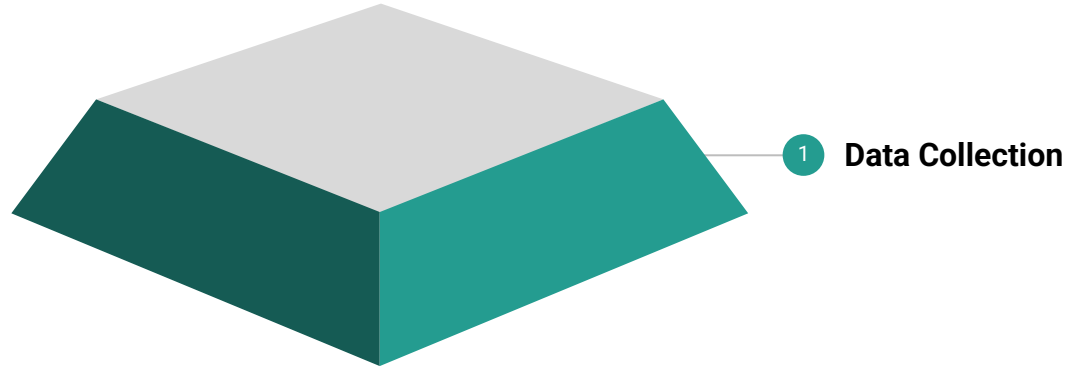
Data-Maturity Model



Typical Data Architecture



Data Collection



Data Wrangling



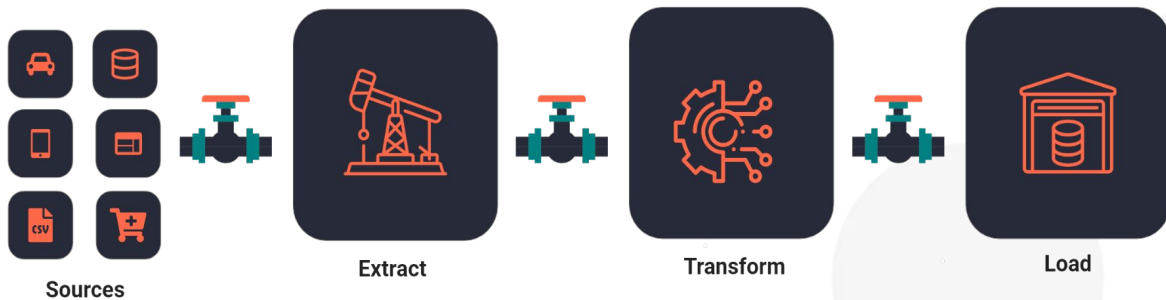
Data Integration



ETL - ELT #

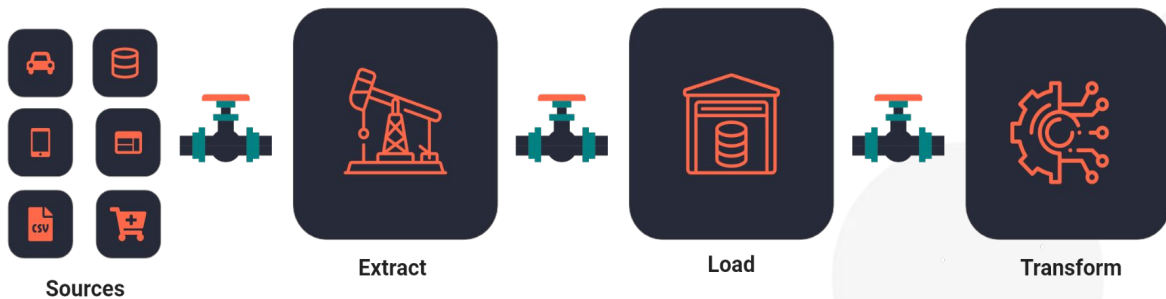
ETL

ETL
Extract, Transform, Load

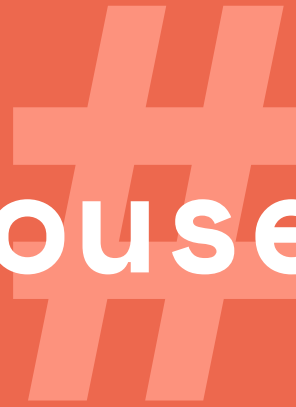


ELT

ELT
Extract, Load, Transform



Data Warehouse



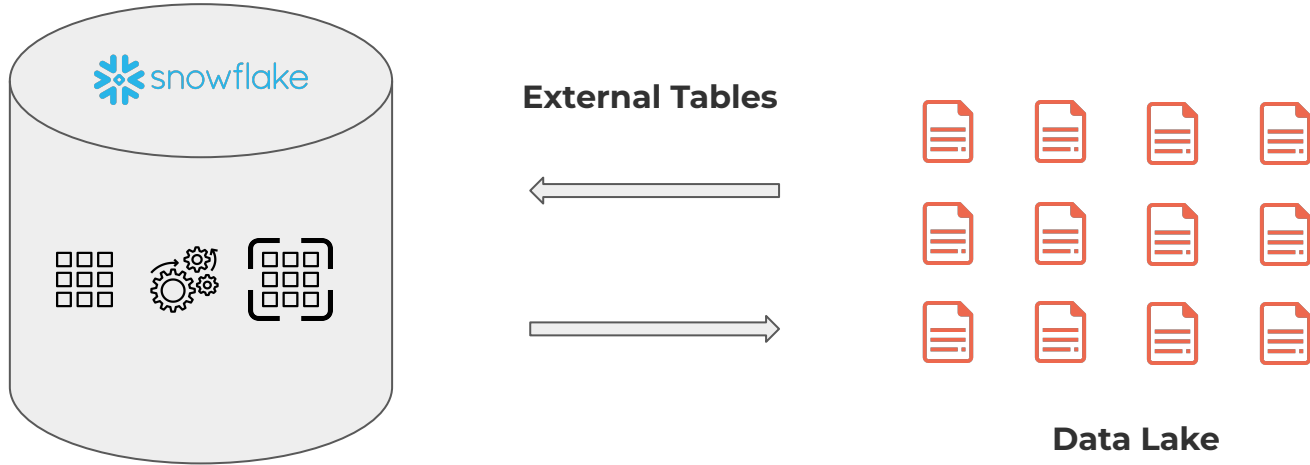
Data Warehouse



External Tables



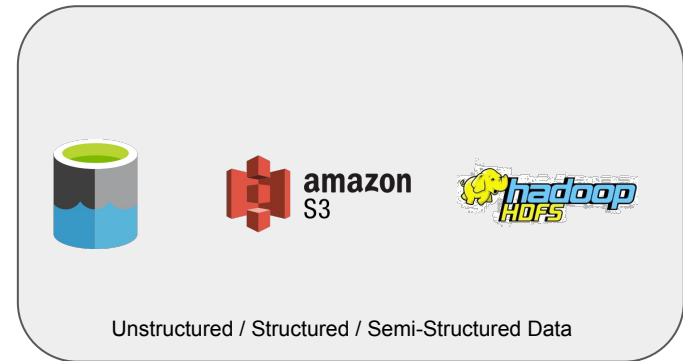
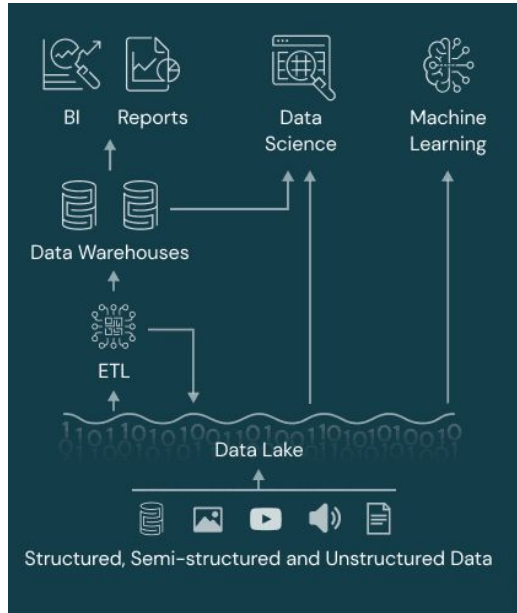
External Tables



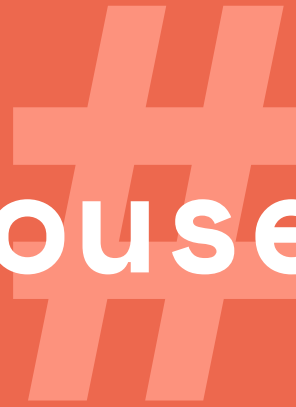
Data Lake



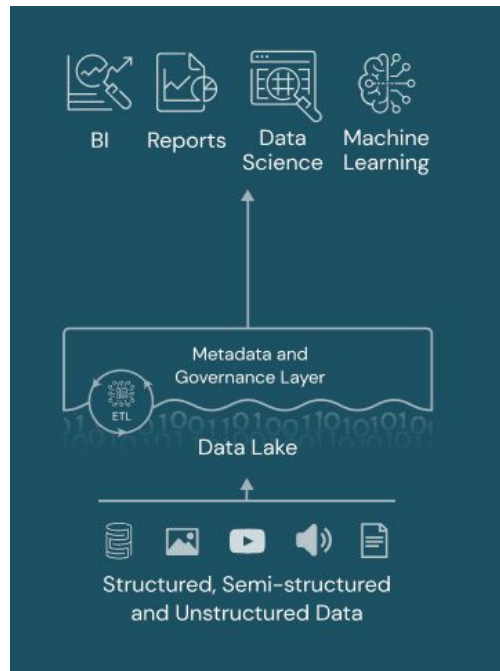
Data Lake



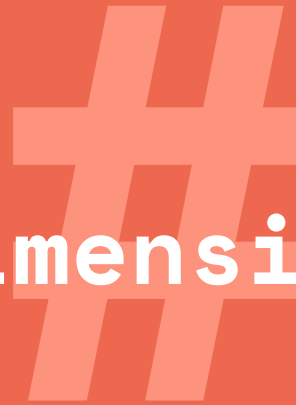
Data Lakehouse



Data Lakehouse



Slowly Changing Dimensions



SCD Type 0

Not updating the DWH table when a Dimension changes

Source			DWH		
Host ID	Name	Fax Number	Host ID	Name	Fax Number
#1	John Doe	1-408-999 8888	#1	John Doe	555-123-4567

SCD Type 1

Updating the DWH table when a Dimension changes, overwriting the original data

No Air-conditioning

Host ID	Air-conditioning
#1	No

Installed Air-conditioning

Host ID	Air-conditioning
#1	Yes

DWH updated

Host ID	Air-conditioning
#1	Yes

DWH

Host ID	Air-conditioning
#1	No

Host ID	Air-conditioning
#1	No

Host ID	Air-conditioning
#1	Yes

SCD Type 2

Keeping full history - Adding additional (historic data) rows for each dimension change

Current rental price (\$300)

Apartment ID	Price
#1	\$300

Change in the rental price (\$450)

Apartment ID	Price
#1	\$450

DWH updated

Source		DWH																						
<table><tr><th>Apartment ID</th><th>Price</th></tr><tr><td>#1</td><td>\$300</td></tr></table>	Apartment ID	Price	#1	\$300		<table><tr><th>Host Key</th><th>Apartment ID</th><th>Price</th><th>Start_Date</th><th>End_Date</th></tr><tr><td>1000</td><td>#1</td><td>\$300</td><td>2020-01-01T00:00:00</td><td>NULL</td></tr></table>	Host Key	Apartment ID	Price	Start_Date	End_Date	1000	#1	\$300	2020-01-01T00:00:00	NULL								
	Apartment ID	Price																						
#1	\$300																							
Host Key	Apartment ID	Price	Start_Date	End_Date																				
1000	#1	\$300	2020-01-01T00:00:00	NULL																				
<table><tr><th>Apartment ID</th><th>Price</th></tr><tr><td>#1</td><td>\$450</td></tr></table>	Apartment ID	Price	#1	\$450	<table><tr><th>Host Key</th><th>Apartment ID</th><th>Price</th><th>Start_Date</th><th>End_Date</th></tr><tr><td>1000</td><td>#1</td><td>\$300</td><td>2020-01-01T00:00:00</td><td>2021-01-01T00:00:00</td></tr></table>	Host Key	Apartment ID	Price	Start_Date	End_Date	1000	#1	\$300	2020-01-01T00:00:00	2021-01-01T00:00:00									
	Apartment ID	Price																						
#1	\$450																							
Host Key	Apartment ID	Price	Start_Date	End_Date																				
1000	#1	\$300	2020-01-01T00:00:00	2021-01-01T00:00:00																				
<table><tr><th>Apartment ID</th><th>Price</th></tr><tr><td>#1</td><td>\$450</td></tr></table>	Apartment ID	Price	#1	\$450		<table><tr><th>Host Key</th><th>Apartment ID</th><th>Price</th><th>Start_Date</th><th>End_Date</th></tr><tr><td>1000</td><td>#1</td><td>\$300</td><td>2020-01-01T00:00:00</td><td>2021-01-01T00:00:00</td></tr><tr><td>1001</td><td>#1</td><td>\$450</td><td>2021-01-01T00:00:00</td><td>NULL</td></tr></table>	Host Key	Apartment ID	Price	Start_Date	End_Date	1000	#1	\$300	2020-01-01T00:00:00	2021-01-01T00:00:00	1001	#1	\$450	2021-01-01T00:00:00	NULL			
	Apartment ID	Price																						
#1	\$450																							
Host Key	Apartment ID	Price	Start_Date	End_Date																				
1000	#1	\$300	2020-01-01T00:00:00	2021-01-01T00:00:00																				
1001	#1	\$450	2021-01-01T00:00:00	NULL																				

SCD Type 2

Keeping full history - Adding additional (historic data) rows for each dimension change

Current rental price (\$300)

Apartment ID	Price
#1	\$300

Change in the rental price (\$450)

Apartment ID	Price
#1	\$450

DWH updated

Source		DWH						
	Apartment ID	Price	Host Key	Apartment ID	Price	Start_Date	End_Date	Is_Current
	#1	\$300	1000	#1	\$300	2020-01-01T00:00:00	9999-12-12T00:00:00	Y
	Apartment ID	Price	Host Key	Apartment ID	Price	Start_Date	End_Date	Is_Current
	#1	\$450	1000	#1	\$300	2020-01-01T00:00:00	9999-12-12T00:00:00	N
	Apartment ID	Price	Host Key	Apartment ID	Price	Start_Date	End_Date	Is_Current
	#1	\$450	1000	#1	\$300	2020-01-01T00:00:00	2021-01-01T00:00:00	N
			1001	#1	\$450	2021-01-01T00:00:00	9999-12-12T00:00:00	Y

SCD Type 3

Keeping limited data history - adding separate columns for original and current value

Listed as Private

Source	DWH										
<table><tr><th>Apartment ID</th><th>Type</th></tr><tr><td>#1</td><td>Private</td></tr></table>	Apartment ID	Type	#1	Private	<table><tr><th>Apartment ID</th><th>Previous Type</th><th>Current Type</th></tr><tr><td>#1</td><td>Private</td><td>Private</td></tr></table>	Apartment ID	Previous Type	Current Type	#1	Private	Private
Apartment ID	Type										
#1	Private										
Apartment ID	Previous Type	Current Type									
#1	Private	Private									
<table><tr><th>Apartment ID</th><th>Type</th></tr><tr><td>#1</td><td>Entire</td></tr></table>	Apartment ID	Type	#1	Entire	<table><tr><th>Apartment ID</th><th>Previous Type</th><th>Current Type</th></tr><tr><td>#1</td><td>Private</td><td>Entire</td></tr></table>	Apartment ID	Previous Type	Current Type	#1	Private	Entire
Apartment ID	Type										
#1	Entire										
Apartment ID	Previous Type	Current Type									
#1	Private	Entire									
<table><tr><th>Apartment ID</th><th>Type</th></tr><tr><td>#1</td><td>Shared</td></tr></table>	Apartment ID	Type	#1	Shared	<table><tr><th>Apartment ID</th><th>Previous Type</th><th>Current Type</th></tr><tr><td>#1</td><td>Entire</td><td>Shared</td></tr></table>	Apartment ID	Previous Type	Current Type	#1	Entire	Shared
Apartment ID	Type										
#1	Shared										
Apartment ID	Previous Type	Current Type									
#1	Entire	Shared									

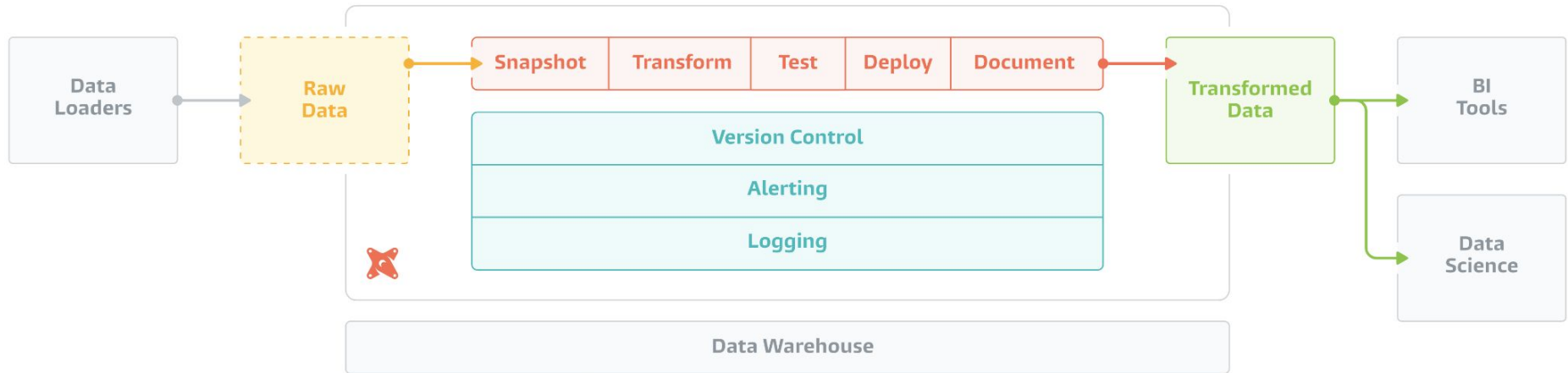
Host changed Private to Entire

Host changed Entire to Shared

dbt Overview



dbt Overview



Analytics Engineering



Common Table Expression (CTE)

A large, stylized orange hashtag symbol (#) is positioned on the right side of the slide, partially overlapping the text "Common Table Expression (CTE)".

CTE

Syntax

SQL ▾

```
WITH <name_of_the_result_set> ([column_names])  
AS  
(  
    <cte_query>  
)  
<reference_the_CTE>
```

CTE

Example

```
-- STEP 1
WITH raw_listings AS (

-- STEP 2
    SELECT * FROM [source].[listings]
)

-- STEP 3
SELECT
    id AS listing_id,
    listing_url,
    name AS listing_name,
    room_type,
    minimum_nights,
    host_id,
    price AS price_str,
    created_at,
    updated_at
FROM raw_listings
```

PROJECT OVERVIEW

Analytics Engineering with Airbnb

ANALYTICS ENGINEERING WITH AIRBNB

- Simulating the life of an Analytics Engineer in Airbnb
- Loading, Cleansing, Exposing data
- Writing test, automations and documentation
- Data source: Inside Airbnb: Berlin

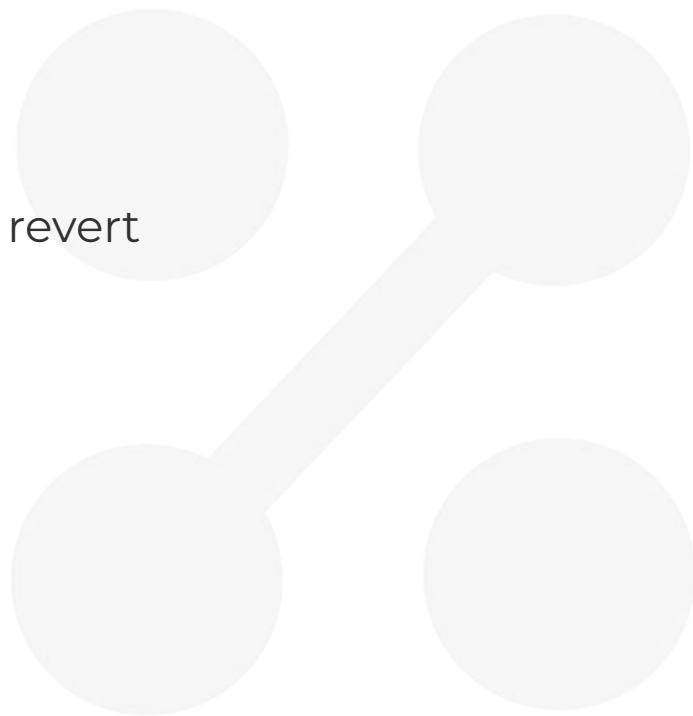


TECH STACK



REQUIREMENTS

- Modeling changes are easy to follow and revert
- Explicit dependencies between models
- Explore dependencies between models
- Data quality tests
- Error reporting
- Incremental load of fact tables
- Track history of dimension tables
- Easy-to-access documentation



NEXT STEPS - SETUP

- Snowflake registration
- Dataset import
- dbt installation
- dbt setup, snowflake connection

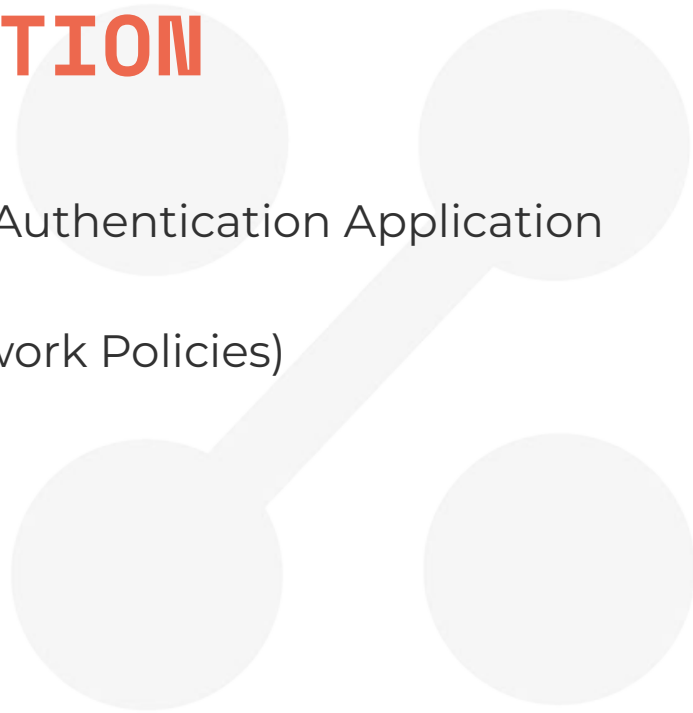
That's it :)



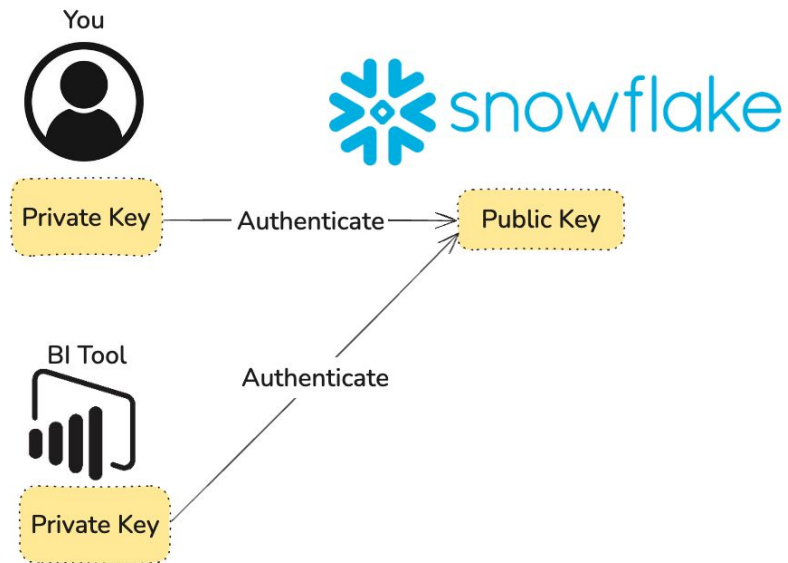
SNOWFLAKE

Trial Account Setup

SNOWFLAKE AUTHENTICATION

- ✗ Username & Password + Multi-Factor Authentication Application
 - ✗ Personal Access Token (Requires Network Policies)
 - ✗ Single Sign-On (SSO)
 - ✓ Keypair-Based Authentication
- 

Public-Key Authentication (PKI)



SNOWFLAKE AUTO-SETUP APP

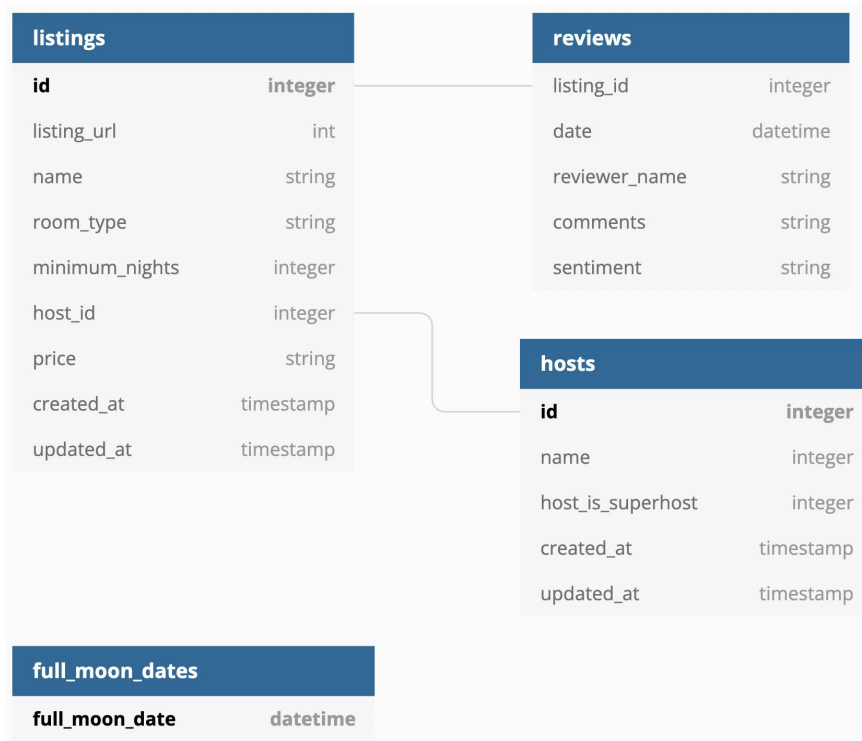
- **Imports the database tables:** listings, hosts, reviews
- **dbt Snowflake user creation** and permissions setup
- **Preset user creation** and permission setup

<https://dbtsetup.nordquant.com>

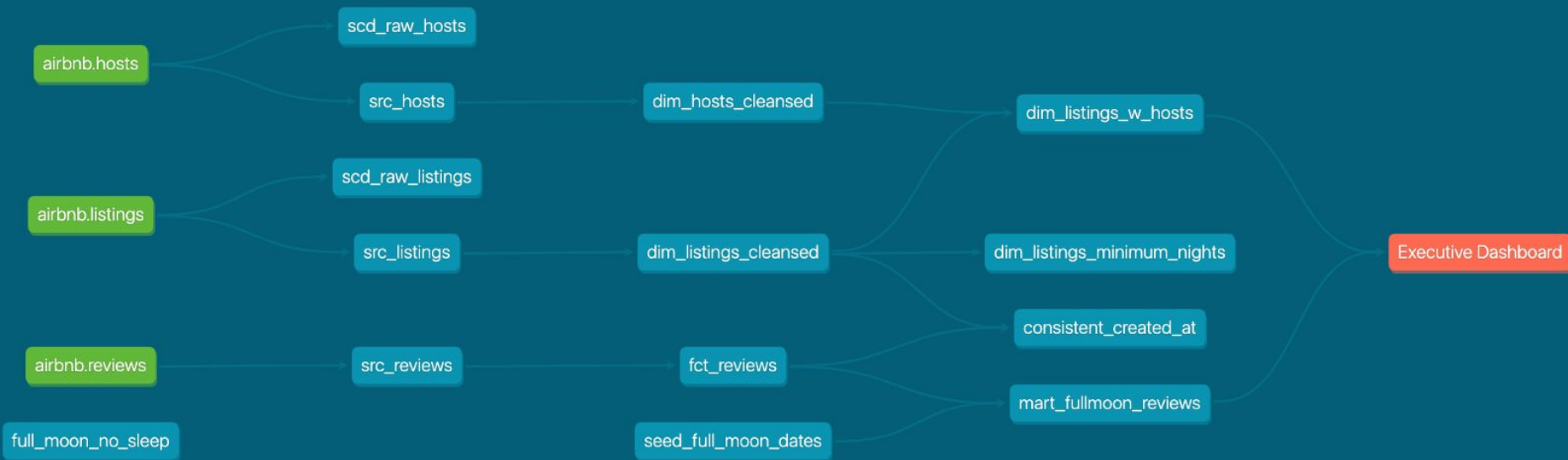
DATA FLOW

Overview

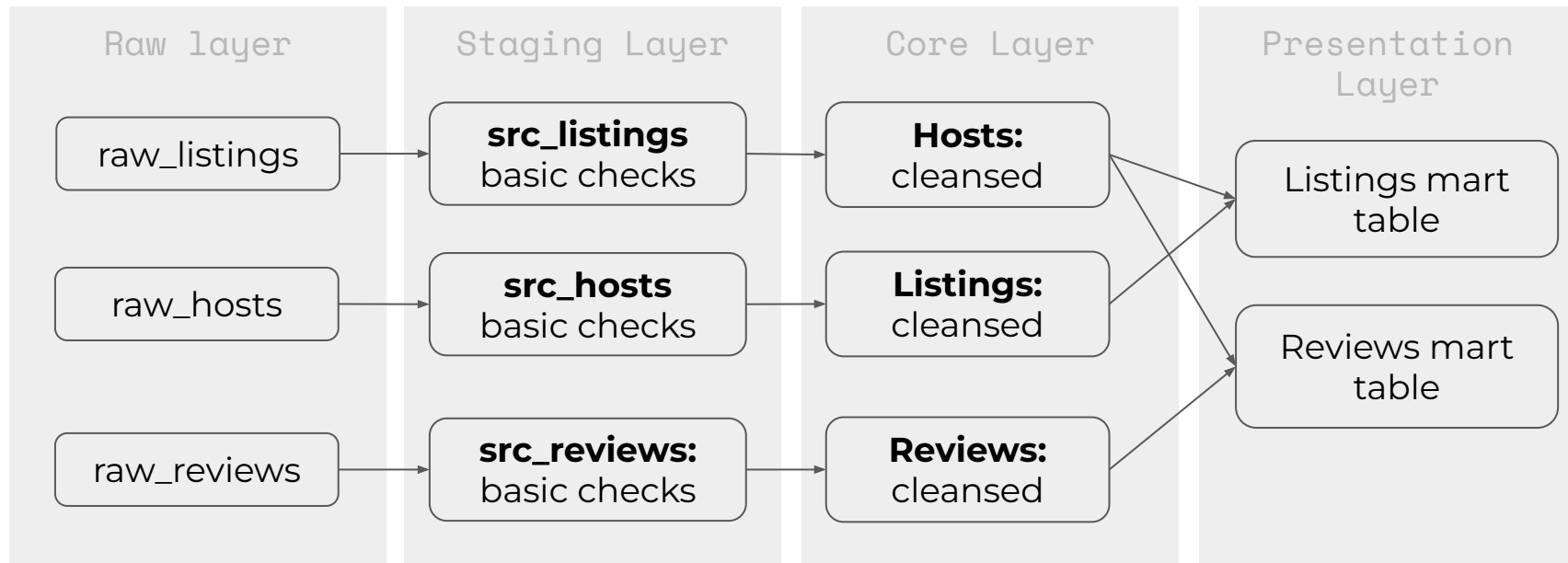
INPUT DATA MODEL



DATA FLOW OVERVIEW



DATA FLOW OVERVIEW



DBT INSTALLATION

Requirements

- Python
- Git
- Github account
- Visual Studio Code
- uv - a Python Package Manager - We'll install it together



Steps

- Fork the Student Repository on GitHub
- Import the Student Repository into Visual Studio Code
- Install uv
- Install dbt



DBT SETUP

dbt init and connecting to Snowflake

SNOWFLAKE CONNECTION ISSUES?

- Ensure VPN is switched off
- Ensure you are not behind a corporate proxy or firewall
- Try to change network or laptop



VSCode Extensions

dbt Visual Studio Code Extensions

The **official VSCode Extension** from dbt Labs.

- Completely built from ground-up for **dbt Fusion**
- This is the **official extension**

The **dbt Power User** extension from ultimate.ai

- Third-party extension
 - Supports both **dbt Core** and **dbt Fusion**
-

Extensions - Feature Comparison

	official dbt Extension	dbt Power User
Price	Free until 15 users requires registration after 14 days	Freemium most features free, more advanced features require a subscription
Official	✓	✗
dbt core Support	✗	✓
dbt Fusion Support	✓	✓
dbt Cloud Platform Support	✓	✓
Code Quality / Helper	✓	✓
Data Lineage	✓	✓
AI Features	✗	✓

Extensions / Now or Later?

- The extension-related courseware assumes that you've completed the core part of the course
- I'd suggest you give the course a shot using **bare-bones Visual Studio Code** to maximise your learning (It will hurt more, I know)
- If you want to try the extensions now, I've created two codespaces for you - these have the code and extensions pre-installed:
 - Official Extension
 - dbt Power User

Codespace links in the Course Resources Repo

MODELS

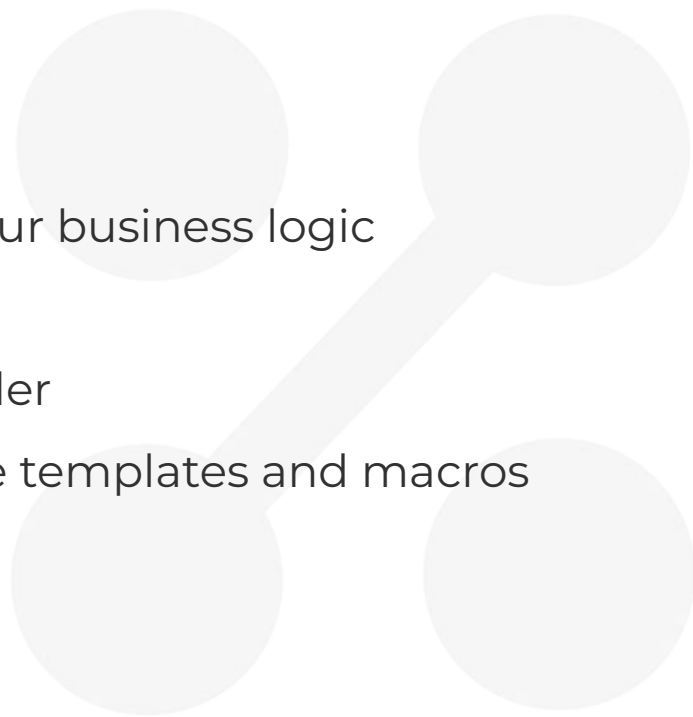
LEARNING OBJECTIVES

- Understand the data flow of our project
- Understand the concept of Models in dbt
- Create three basic models:
 - src_listings
 - src_reviews: guided exercises
 - src_hosts: individual lab

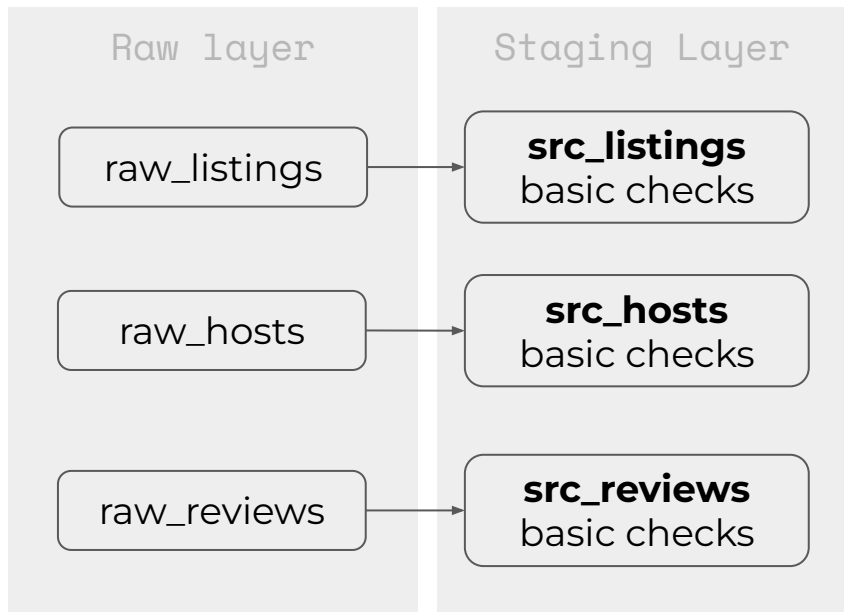


MODELS OVERVIEW

- Models are the basic building block of your business logic
- Materialized as tables, views, etc...
- They live in SQL files in the `models` folder
- Models can reference each other and use templates and macros



DATA FLOW PROGRESS



GUIDED EXERCISE

src_reviews.sql

Create a new model in the ``models/src/`` folder called ``src_reviews.sql``.

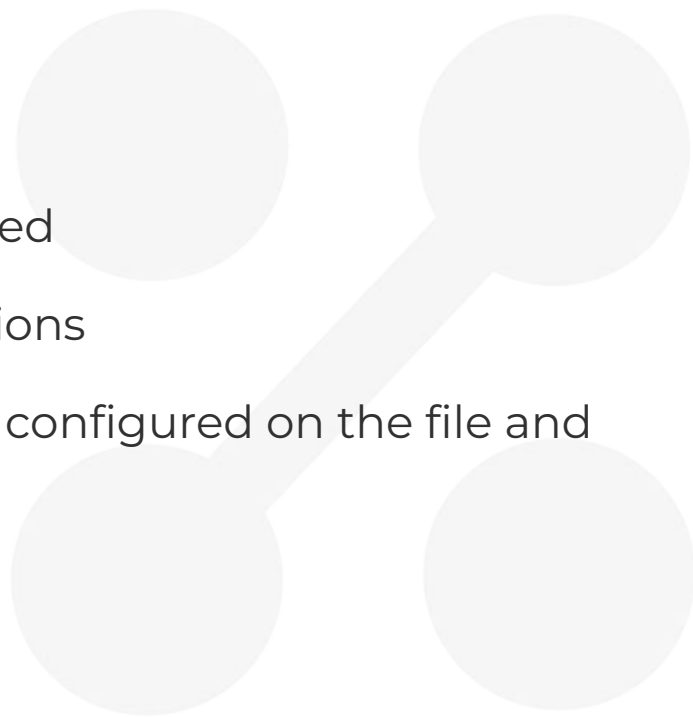
- Use a CTE to reference the AIRBNB.RAW.RAW_REVIEWS table
- SELECT every column and every record, and rename the following columns:
 - *date* to *review_date*
 - *comments* to *review_text*
 - *sentiment* to *review_sentiment*
- Execute ``dbt run`` and verify that your model has been created

(You can find the solution among the resources)

MATERIALIZATIONS

LEARNING OBJECTIVES

- Understand how models can be connected
- Understand the four built-in materializations
- Understand how materializations can be configured on the file and project level
- Use *dbt run* with extra parameters



MATERIALIZATIONS

MATERIALISATIONS OVERVIEW

View

Use it

- You want a lightweight representation
- You don't reuse data too often

Don't use it

- You read from the same model several times

Table

Use it

- You read from this model repeatedly

Don't use it

- Building single-use models
- Your model is populated incrementally

Incremental (table appends)

Use it

- Fact tables
- Appends to tables

Don't use it

- You want to update historical records

Ephemeral (CTEs)

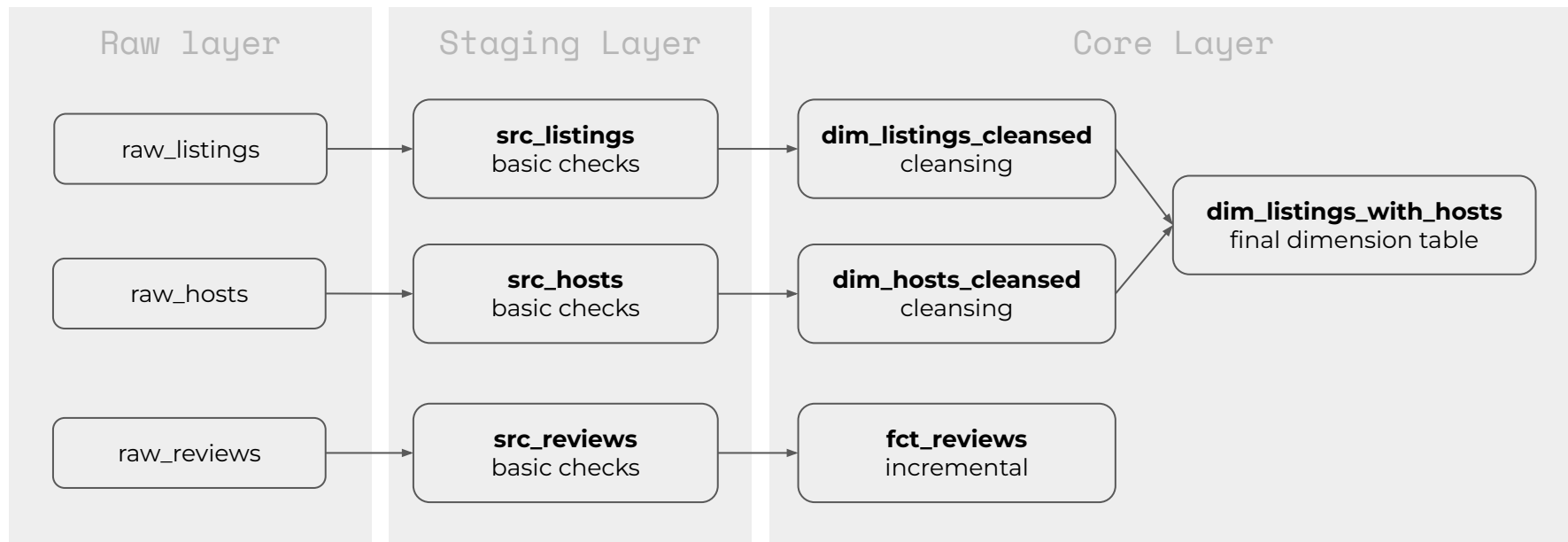
Use it

- You merely want an alias to your data

Don't use it

- You read from the same model several times

DATA FLOW PROGRESS



GUIDED EXERCISE

dim_hosts_cleansed.sql

Create a new model in the ``models/dim/`` folder called ``dim_hosts_cleansed.sql``.

- Use a CTE to reference the ``src_hosts`` model
- SELECT every column and every record, and add a cleansing step to *host_name*:
 - If *host_name* is not null, keep the original value
 - If *host_name* is null, replace it with the value `'Anonymous'`
 - Use the `NVL(column_name, default_null_value)` function
- Execute ``dbt run`` and verify that your model has been created

(You can find the solution among the resources)

INCREMENTAL STRATEGIES

append

Pros

- Simplest
- Fully-transparent logic

Cons

- No duplicate checks

merge

Pros

- Idempotent (no duplicates)
- Updates existing records
- Snowflake/BigQuery default

Cons

- Slower
- Requires unique_key

delete+insert

Pros

- Works on DWHs with no MERGE support
- Simple mental model

Cons

- Two operations
- Requires unique_key

insert_overwrite

Use it

- Fast for partitioned tables

Cons

- Requires Partitioned table
- All or nothing (replaces whole partitions)

+microbatch

SOURCES & SEEDS

LEARNING OBJECTIVES

- Understand the difference between seeds and sources
- Understand source-freshness
- Integrate sources into our project



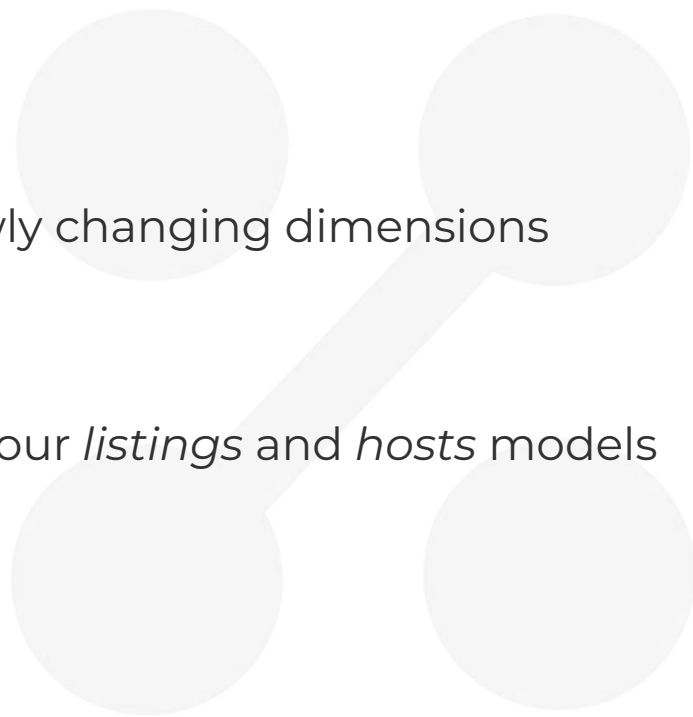
SOURCES AND SEEDS OVERVIEW

- Seeds are local files that you upload to the data warehouse from dbt
 - Sources is an abstraction layer on the top of your input tables
 - Source freshness can be checked automatically
- 
-

SNAPSHOTS

LEARNING OBJECTIVES

- Understand how dbt handles type-2 slowly changing dimensions
- Understand snapshot strategies
- Learn how to create snapshots on top of our *listings* and *hosts* models



SNAPSHOTS

Overview

TYPE-2 SLOWLY CHANGING DIMENSIONS

host_id	host_name	email
1	Alice	alice.airbnb@gmail.com
2	Bob	bob.airbnb@gmail.com

TYPE-2 SLOWLY CHANGING DIMENSIONS

host_id	host_name	email
1	Alice	alice.airbnb@gmail.com
2	Bob	bobs.new.address@gmail.com

TYPE-2 SLOWLY CHANGING DIMENSIONS

host_id	host_name	email	dbt_valid_from	dbt_valid_to
1	Alice	alice.airbnb@gmail.com	2022-01-01 00:00:00	null
2	Bob	bob.airbnb@gmail.com	2022-01-01 00:00:00	2022-03-01 12:53:20
3	Bob	bobs.new.address@gmail.com	2022-03-01 12:53:20	null

CONFIGURATION AND STRATEGIES

- Snapshots live in the *snapshots* folder
 - Strategies:
 - *Timestamp*: A **unique key** and an **updated_at** field is defined on the source model. These columns are used for determining changes.
 - *Check*: Any change in a set of columns (or all columns) will be picked up as an update.
-

GUIDED EXERCISE

raw_hosts_snapshot.yml

Create a new snapshot in the ``snapshots/`` folder called ``raw_hosts_snapshot.yml``.

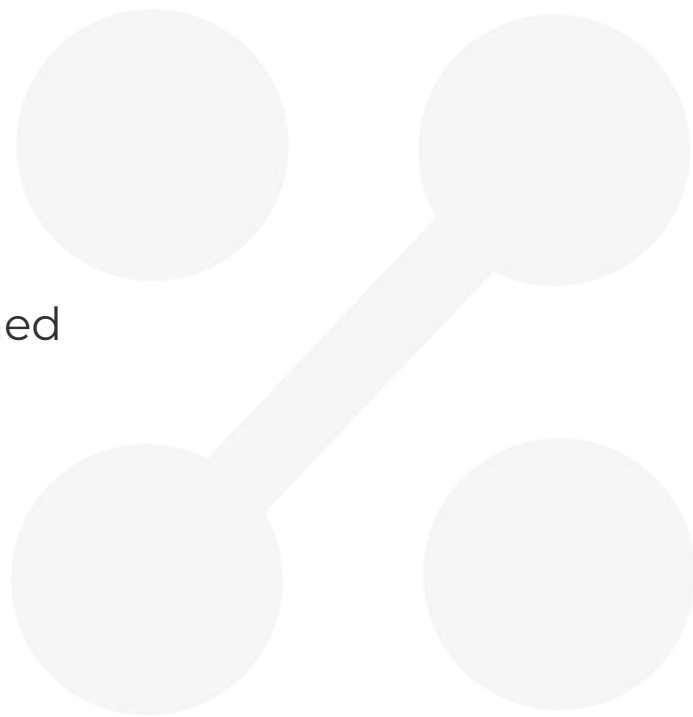
- Call it `scd_raw_hosts`
- Set the target table name to the *hosts* source table
- Use the *timestamp* strategy, figure out the unique key and `updated_at` column to use
- Invalidate hard deletes
- Execute ``dbt snapshot`` and verify that your snapshot has been created

(You can find the solution among the resources)

TESTS

LEARNING OBJECTIVES

- Understand **Unit Tests** and **Data Tests**
- Understand how these tests can be defined
- Configure built-in generic Data Tests
- Create your own singular Data Tests
- Create Unit Tests for your data

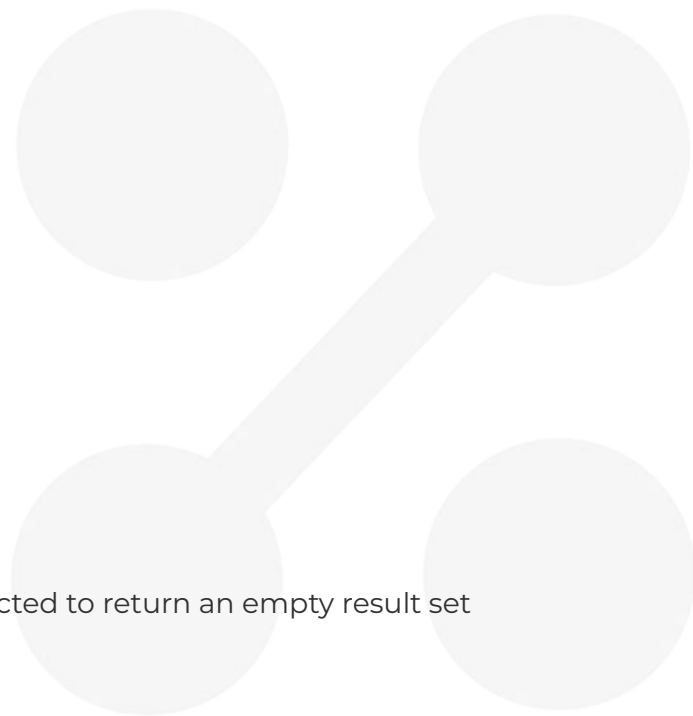


TESTS OVERVIEW

- dbt has two types of tests: **Unit Tests** and **Data Tests** and it supports **Contracts**
- **Unit Tests** test transformations with a small sample of mock data you provide
- **Data Tests** test data integrity and quality on the actual data
- **Contracts** enforce the schema of models
(such as column names, types and constraints)

DATA TESTS OVERVIEW

- There are two types of data tests: **singular** and **generic**
- There are four built-in generic tests:
 - unique
 - not_null
 - accepted_values
 - Relationships
- Singular data tests are SQL queries stored in tests which are expected to return an empty result set
- *You can define your own custom generic tests*
- *You can import tests from dbt packages (will discuss later)*



GUIDED EXERCISE

TEST `dim_hosts_cleansed`

Create a generic data test for the ``dim_hosts_cleansed`` model.

- *host_id*: Unique values, no nulls
- *host_name* shouldn't contain any null values
- *is_superhost* should only contain the values *t* and *f*.
- Execute ``dbt test`` to verify that your tests are passing
- **Bonus: Figure out which tests to write for ``fct_reviews`` and implement them**

(You can find the solution among the resources)

CONSTRAINTS

- Run tests at **insert time**
 - An **enforced contract** must be defined to make constraints work.
 - Only works on **table** and **incremental** materializations
 - Functionality heavily depends on platform. Theoretically you can test for *primary_key, foreign key, check, not_null, unique*
 - Only a fraction of these are enforced in **Snowflake** at the time of recording: **not_null**
-

JINJA AND MACROS

LEARNING OBJECTIVES

- Understand the basic building blocks and control structures of Jinja
- Use macros to add new functionality to your project

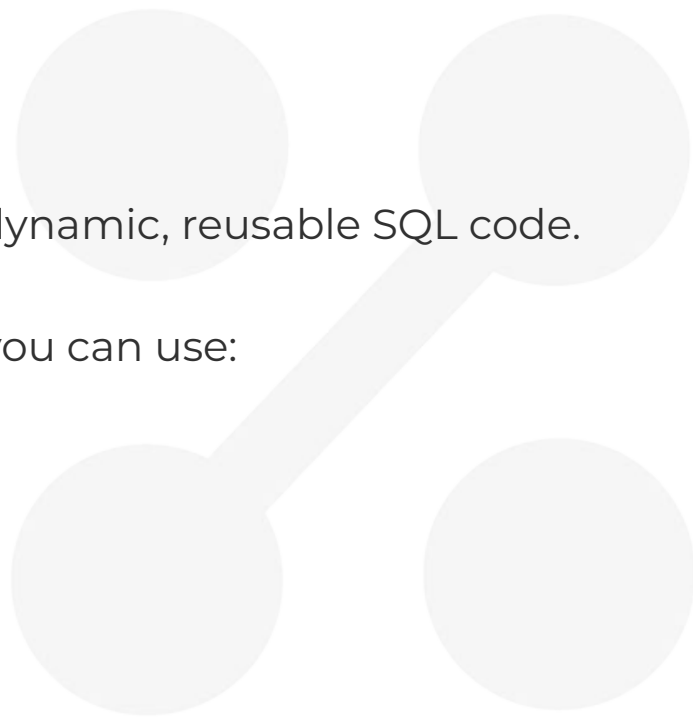


JINJA

- A **templating language** that lets you write dynamic, reusable SQL code.

Think of it as "**programming inside your SQL**" - you can use:

- Variables
 - Conditionals and Loops
 - Functions
 - Macros
-



JINJA - BASIC BUILDING BLOCKS

- Comments: `{# This is a comment #}`
 - *Instantly removed - won't appear anywhere*
- Statements: `{% set my_name = "Zoltan" %}`
- Expressions: `{{ my_name }}`
 - *Outputs "Zoltan"*

(SOME) BUILT-IN VARIABLES IN DBT

- `{{ this }}`
 - References the model being built
 - `{{ target.name }}`
 - The name of our target - the default is defined in *dbt_profiles.yml* (dev)
 - `{{ target.database }}`
 - The target database
 - `{{ target.schema }}`
 - The target schema
-

JINJA - MACRO (FUNCTION) EXAMPLES

- `{{ log("Debug message", info=True) }}` Prints to the console
 - `{{ var("user_name") }}` Get variable value
 - `{{ ref('dim_listings_cleansed') }}` References another model - **dbt specific**
 - `{{ source('airbnb', 'listings') }}` References a source table - **dbt specific**
 - `{{ config('dim_listings_cleansed') }}` References another model - **dbt specific**
 - `{{ adapter.get_columns_in_relation }}` Adapter-level functionality - **dbt specific**
-

MACROS (FUNCTIONS) IN DBT

- Macros are jinja macros created in the **macros** folder
- There are many built-in macros in DBT
- You can use macros in various places, such as:
 - model definitions
 - tests
 - other macros



CUSTOM BUILDING BLOCKS IN DBT

- { % **test** ... % } Test Definition
- { % **snapshot** % } Snapshot definition
- { % **docs** % } Documentation block
- { % **materialization** % } Custom materialization

JINJA - DEFINING MACROS

```
{% macro select_positive_values(model, column_name) %}
```

```
SELECT *
```

```
FROM {{ model }}
```

```
WHERE {{ column_name }} > 0
```

```
{% endmacro %}
```



JINJA - IF/ELSE

```
SELECT * FROM {{ ref('src_reviews') }} WHERE review_text IS NOT NULL
```

```
{% if is_incremental() %}
```

```
    AND review_date > (SELECT MAX(review_date) FROM {{ this }})
```

```
    {{ log('Loading incrementally - new reviews only', info=True) }}
```

```
{% else %}
```

```
    {{ log('Loading all reviews - first run or full refresh', info=True) }}
```

```
{% endif %}
```



JINJA - LOOPS

```
{% set columns = ['listing_id', 'listing_name', 'price'] %}
```

```
SELECT
```

```
{% for col in columns %}
```

```
  {{ col }}{% if not loop.last %}, {% endif %}
```

```
{% endfor %}
```

```
FROM {{ ref('dim_listings_cleansed') }}
```



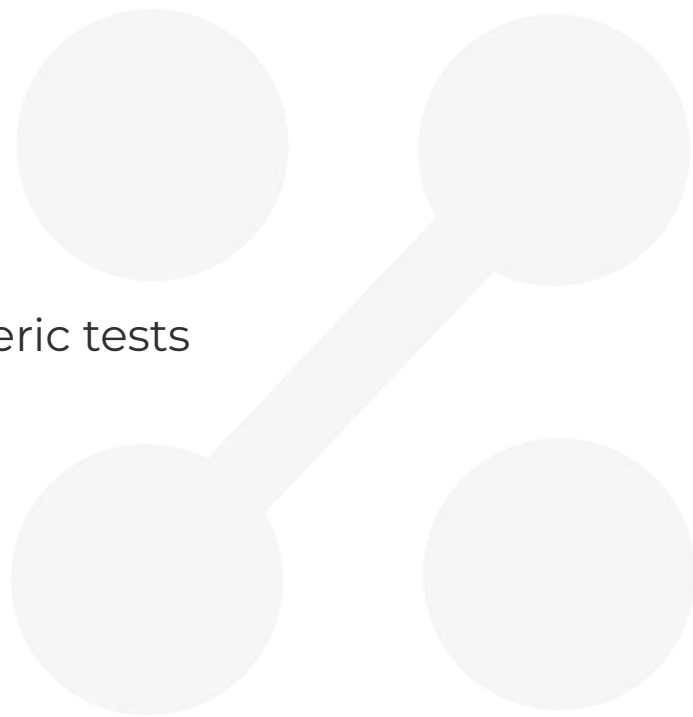
WHITESPACE CONTROL

- `{% statement %}`: **No** whitespace control
- `{%- statement %}`: Strip whitespace **BEFORE**
- `{% statement -%}`: Strip whitespace **AFTER**
- `{%- statement -%}` Strip whitespace **BEFORE** and **AFTER**

JINJA, CUSTOM TESTS AND PACKAGES

LEARNING OBJECTIVES

- Understand how macros are created
- Use macros to implement your own generic tests
- Find and install third-party dbt packages



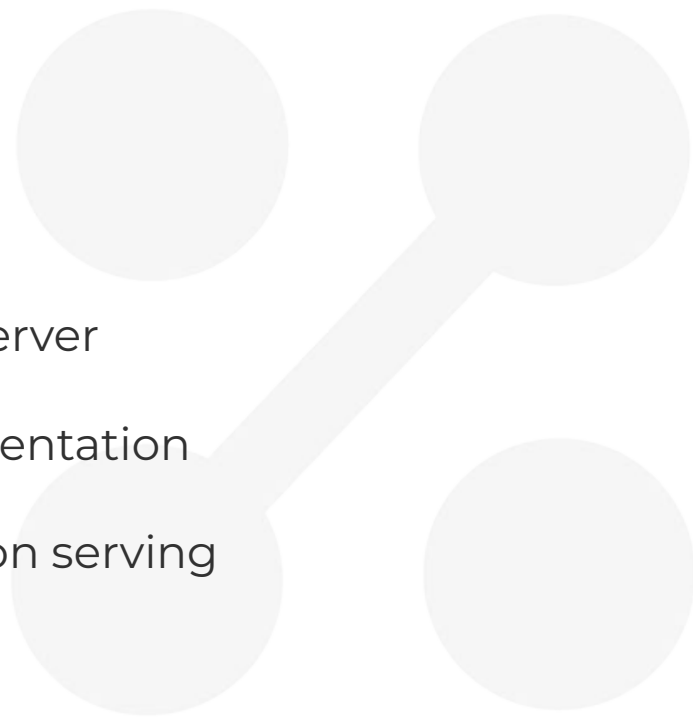
MACROS, CUSTOM TESTS AND PACKAGES

- Macros are jinja templates created in the *macros* folder
 - There are many built-in macros in DBT
 - You can use macros in model definitions and tests
 - A special macro, called *test*, can be used for implementing your own generic tests
 - dbt packages can be installed easily to get access to a plethora of macros and tests
-

DOCUMENTATION

LEARNING OBJECTIVES

- Understand how to document models
- Use the documentation generator and server
- Add assets and markdown to the documentation
- Discuss dev vs. production documentation serving

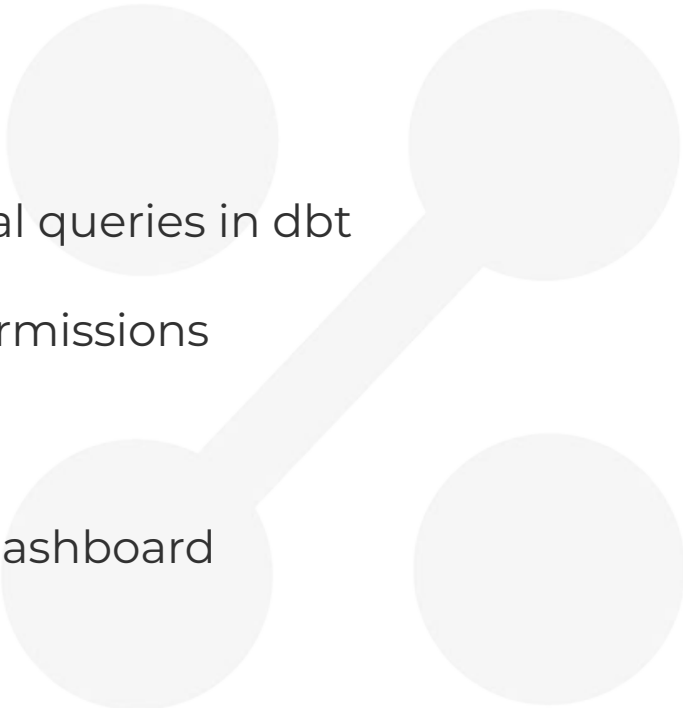


DOCUMENTATION OVERVIEW

- Documentations can be defined two ways:
 - In yaml files (like *schema.yml*)
 - In standalone markdown files
 - Dbt ships with a lightweight documentation web server
 - For customizing the landing page, a special file, *overview.md* is used
 - You can add your own assets (like images) to a special project folder
-

ANALYSES, HOOKS AND EXPOSURES

LEARNING OBJECTIVES

- Understand how to store ad-hoc analytical queries in dbt
 - Work with dbt hooks to manage table permissions
 - Build a dashboard in Preset
 - Create a dbt exposure to document the dashboard
- 

HOOKS

- Hooks are SQLs that are executed at predefined times
 - Hooks can be configured on the project, subfolder, or model level
 - Hook types:
 - `on_run_start`: executed at the start of *dbt {run, seed, snapshot}*
 - `on_run_end`: executed at the end of *dbt {run, seed, snapshot}*
 - `pre-hook`: executed before a model/seed/snapshot is built
 - `post-hook`: executed after a model/seed/snapshot is built
-

DEBUGGING WORKFLOWS

Commands for debugging

Command	dbt Core	dbt Fusion
<code>dbt --version</code>	dbt command available?	
<code>dbt debug</code>	Warehouse connection	
<code>dbt clean</code>	Clear target folder	
<code>dbt ls</code>	What models can dbt see	
<code>dbt deps</code>	Packages available?	
<code>dbt parse</code>	Validate YAML, write <code>manifest.json</code>	
dbt compile	Render Jinja -> SQL	Also checks SQL syntax
<code>dbt run</code>	Run/Materialize the model	
<code>dbt test</code>	Run tests	
<code>dbt show</code>	What would a model produce?	
<code>dbt build</code>	Runs <code>dbt seed</code> , <code>dbt run</code> , <code>dbt snapshot</code> , <code>dbt test</code> in order	

Commands for debugging

How can we make debugging the full pipeline faster?

Command	dbt Core	dbt Fusion
<code>dbt run --empty</code>	Practically executed <code>dbt run</code> with LIMIT 0 on all models	
<code>dbt run --sample <time expression></code>	Samples data based on a time range	

Selectors

By name	<code>-s model_name, -s path</code>	Select a model or path
Graph Operators	<code>+model, model+, @model</code>	Selects a subgraph from DAG
Methods	<code>tag:, source:, path:</code>	Selects by type or other property
State	<code>state:new, state:modified</code>	Comparing my DAG to another DAG
Result	<code>Result:error, result:fail, ...</code>	Rerun only what failed (tests, models, etc)
Set operators	<code><s-a> <s-b>, <s-a>, <s-b></code>	Union or Intersection of selectors
Exclusion	<code>-exclude <s></code>	Exclude models included in selector
“Selectors”	Combining selectors in YAML	Specifying common selector criteria

PYTHON MODELS

Python Models

Expressing a transformation as **Python code**

- Only necessary for a small, advanced set of use cases.
- The code runs on the target platform
- A subset of data platforms supported, i.e.:
 - Databricks
 - Snowflake
 - BigQuery
- Limitations:
 - Usually much slower as SQL transformations
 - Syntax not platform-agnostic



Python Models - syntax

```
def model(dbt, session):  
    dt.ref("model_name") # referencing models  
    # Your transformations  
    # returns either a Pandas or platform-specific DataFrame  
    return df
```

dbt: dbt Functionalities (refs, sources, variables, ...)

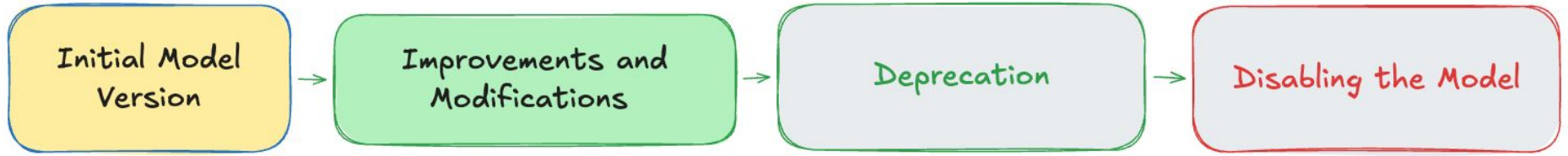
session: Target-platform-specific functionalities (i.e. accessing built-in functions)

Advanced Python Models

- Materialization can be specified (table and incremental)
 - view and ephemeral materializations not supported
- **Python packages** can be requested to be installed
- **dbt configuration, variables**, etc accessible through the `dbt` object
- **Custom SQL** can be executed within from within the function

MODEL LIFECYCLE

Model Lifecycle



Model Lifecycle

- Initial Model Version: You know this already :)
- Improvements / Modifications:
 - Modify in-place (⚠ may break downstream models)
 - Use **Model Versions**
- Deprecation: the **deprecated** configuration
- Disabling a model: the **enabled** configuration



Model Versioning

Problem:

- Changing the schema of a model is possible **breaking downstream models**, that are **managed by others**

dbt's Solution

- Create a new version of the same *model*, keep the old.
 - Implement a flag to indicate the old *model* should be used by default
 - Downstream model updates reference the a specific version of the *model* explicitly
-

Model Versioning - File Names

Filename	Comment
dim_hosts_cleansed_v1.sql	
dim_hosts_cleansed_v2.sql	
dim_hosts_cleansed.sql	Use the <code>defined_in</code> attribute

dbt IN PRODUCTION

Slim CI - Learning Objectives

- Working with different **target** warehouses and **schemas**
 - Using **Environmental Variables**
 - Working with **state** (new, modified, failed)
 - **--defer and --clone**
 - Implementing **Slim CI** through Pull Requests
- 
-

Environment Variables

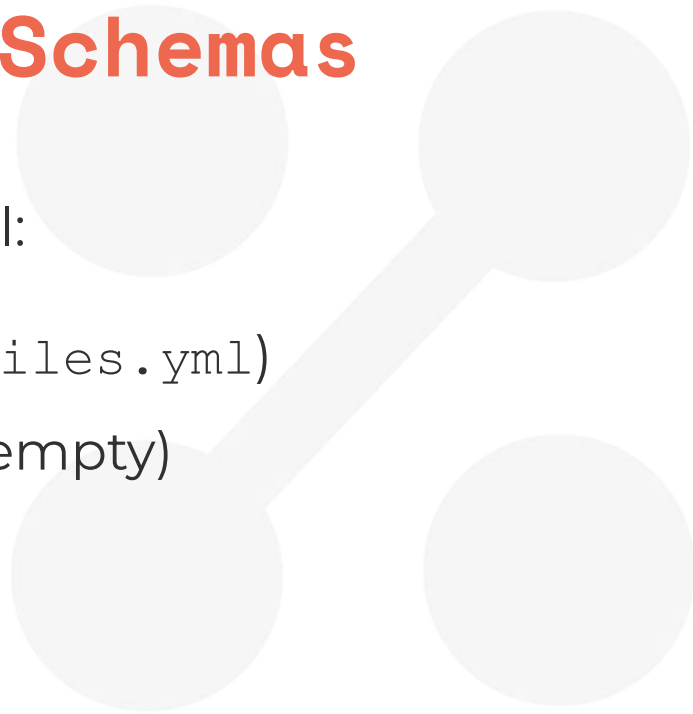
Passing values from the running environment (command line, CI tool, etc). **Not the same as variables**

```
{{ env_var('VARIABLE_NAME', 'default value') }}
```

Working with Custom Schemas

Two schema configs affect each model:

- Target schema (take a look at `profiles.yml`)
- Model's schema (by default this is empty)



Custom Schemas

By default schema = `target.schema + _ + custom schema name`:

I.e. if the model name is `dim_listings`

target.schema	custom schema name	Actual schema built
DEV	none	DEV.dim_listings
DEV	MYSHEMA	DEV_MYSHEMA.dim_listings
PROD	MYSHEMA	PROD_MYSHEMA.dim_listings

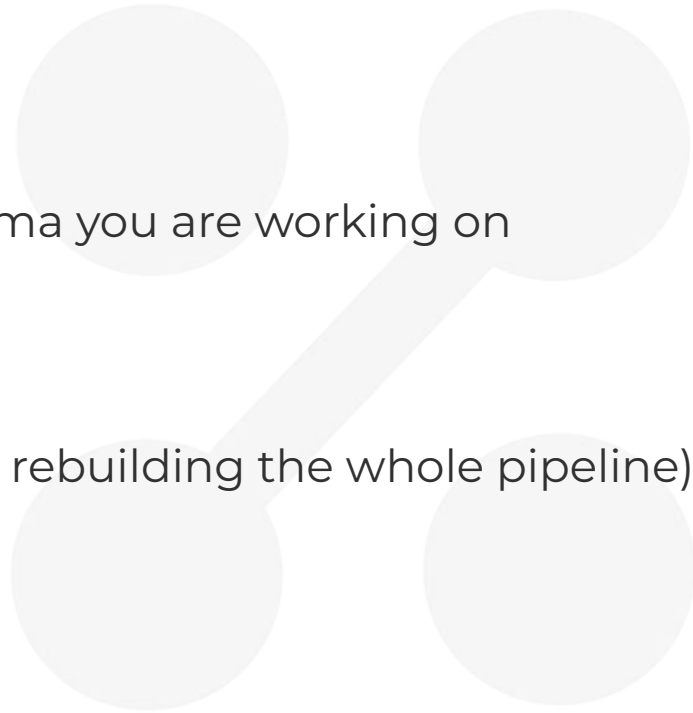
Confusing? `generate_schema_name` macro to the rescue!

Overriding the Schema

After implementing a production-grade configuration in `generate_schema_name.sql`

target.name	custom schema name	Actual schema built
prod	none	PROD.dim_listings
prod	MART	MART.dim_listings
staging	none	STAGING.dim_listings
staging	MART	STAGING_MART.dim_listings
myfeature	none	DBT_MYFEATURE.dim_listings
myfeature	MART	DBT_MYFEATURE_MART.dim_listings

Basic Concepts

- **Target:** The warehouse / database / schema you are working on
 - **CI:** Test every change
 - **CD:** Ship/deploy every change
 - **Slim CI:** test only what changed (without rebuilding the whole pipeline)
- 

dbt in Production - Slim CI

Problem

Rebuilding every model for each pull request can be slow

Solution

Materialize / test only models that have changed compared to the target branch of the PR

Use production tables as stand-ins for unmodified tables

Working with state

dbt stores state (model build state, source freshness, etc) **in json files**

- State is stored in **target/** by default (change this with **--target-path**)
- Point dbt to another state folder with the **--state** parameter

Use case:

- 1) **Store production state** in a folder (possible in a cloud storage)
 - 2) **Make your changes** (yaml, SQL, etc.)
 - 3) Point dbt to the production state so it knows what has been changed
And **only rebuild what's necessary**
-

Working with the state selector

Use the `state:` parameter to

new	build new models only
modified	Build models modified (compared to target state)
<code>modified.body</code>	SQL body modified
<code>modified.config</code>	Config modified
...	Multiple selectors for fine-tuning state

--defer vs. --clone

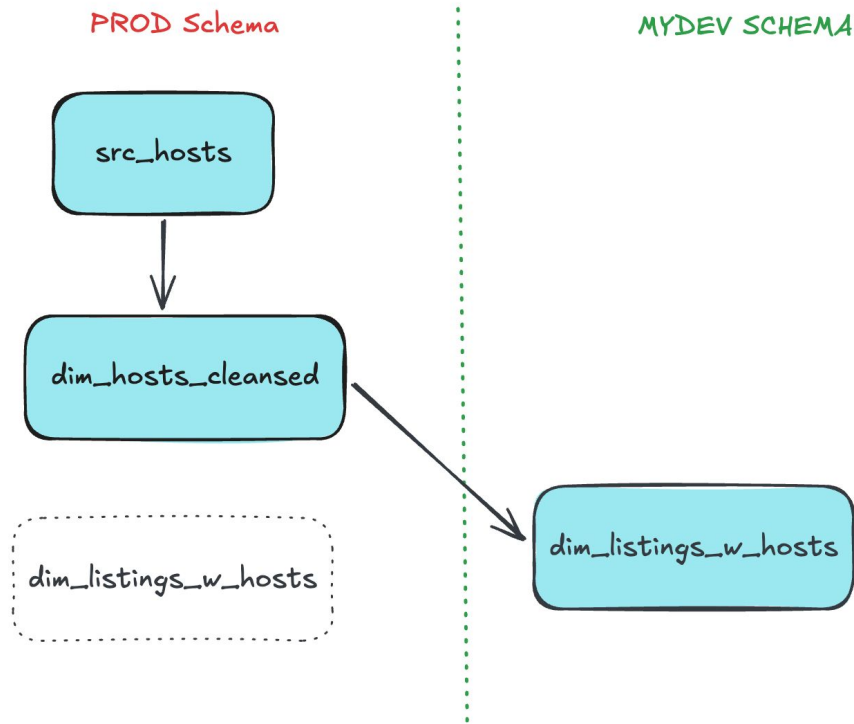
Use --defer:

- You want the cheapest possible option in CI
- Doesn't materialize *any model* besides the ones modified on the dev's schema (it uses the "prod" schema as a stand-in for missing models)

Use --clone

- You want a cheap full copy of the "prod" schema
 - Testing incremental models (defer would need --full-refresh)
 - Deployment: clone prod system to staging, test, deploy only if tests pass
 - Seeding dev environments
-

--defer



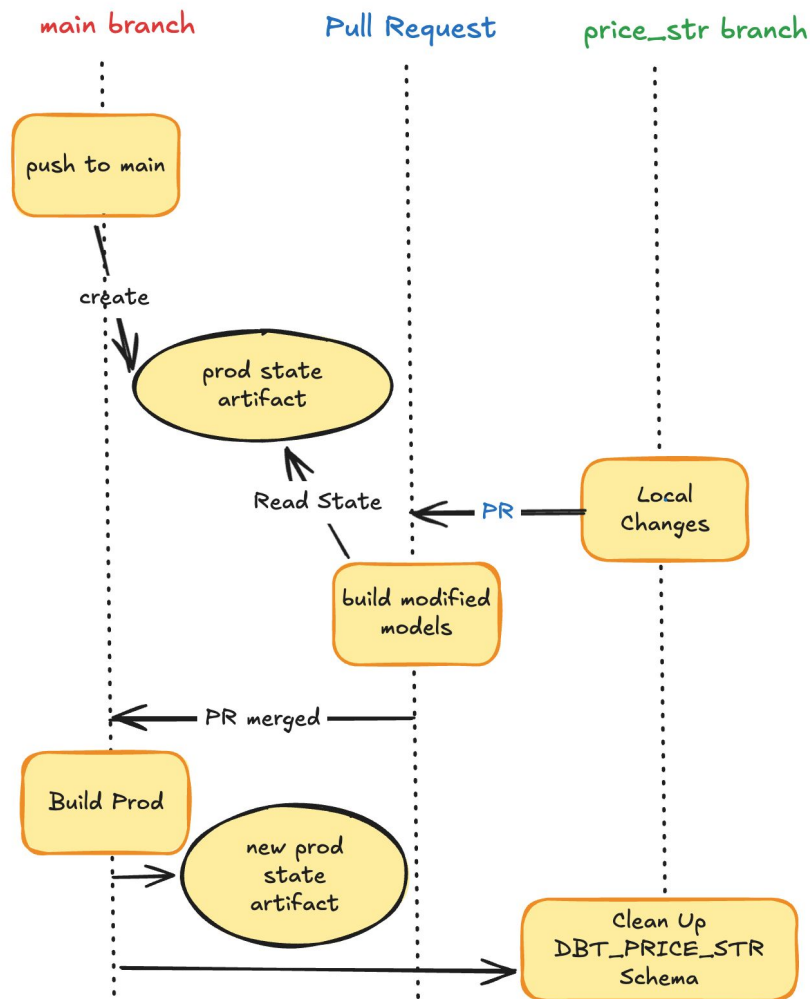
Git Concepts (oversimplified)

- **Commit:** save a snapshot of your changes locally
 - **Push:** send your commits to the central repo
 - **Branch:** An independent line of work
 - `main`: production-ready, deployed code
 - feature branches: where analytics engineers do their work
 - Pull Request (**PR**): propose merging your branch into `main`
-

Slim CI

Three Workflows

- Push to *main*: build everything and create *prod-target* artifact
- Pull Request: *build state:modified*
- Merged Pull Request:
 - Implicit push to *main*
 - Clean up development schema



dbt Fusion

&

The Official VSCode Extension

A new Engine for dbt

- Completely rewritten from the ground up in Rust
 - Magnitudes-Faster Parsing
 - “Code Comprehension Engine”
- Released as Source-available from dbt Labs
- Released along an Official Visual Studio Code Extension

Parsing a 10,000 model project

Cold parse, Rust	-	0.00s
Partial parse, Python	—	0.00s
Cold parse, Python	—————	0.00s

LEARNING OBJECTIVES

- Understand how the Parser and Compiler work
- Feature Matrix - dbt Fusion vs dbt Core
- License differences
- Get hands-on with dbt Fusion and the VSCode Extension

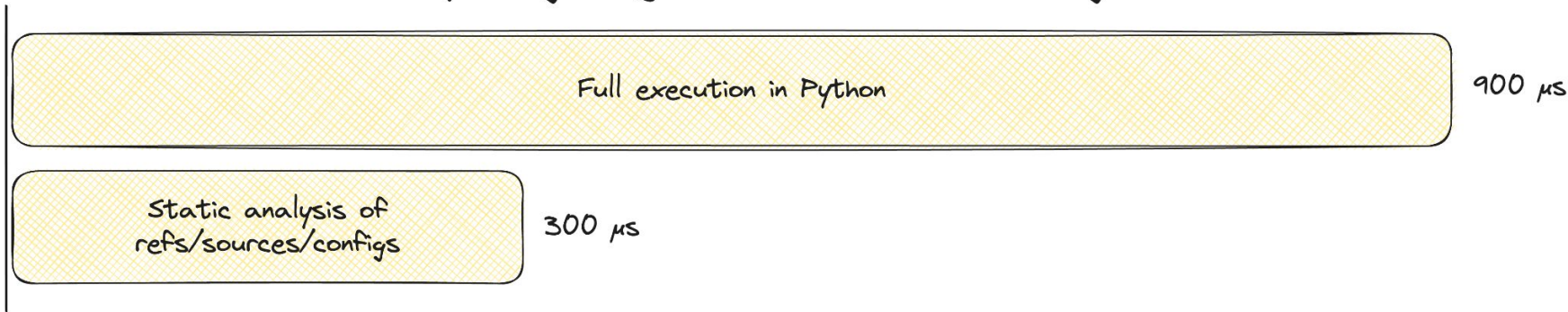


dbt Fusion

Technical Deep Dive

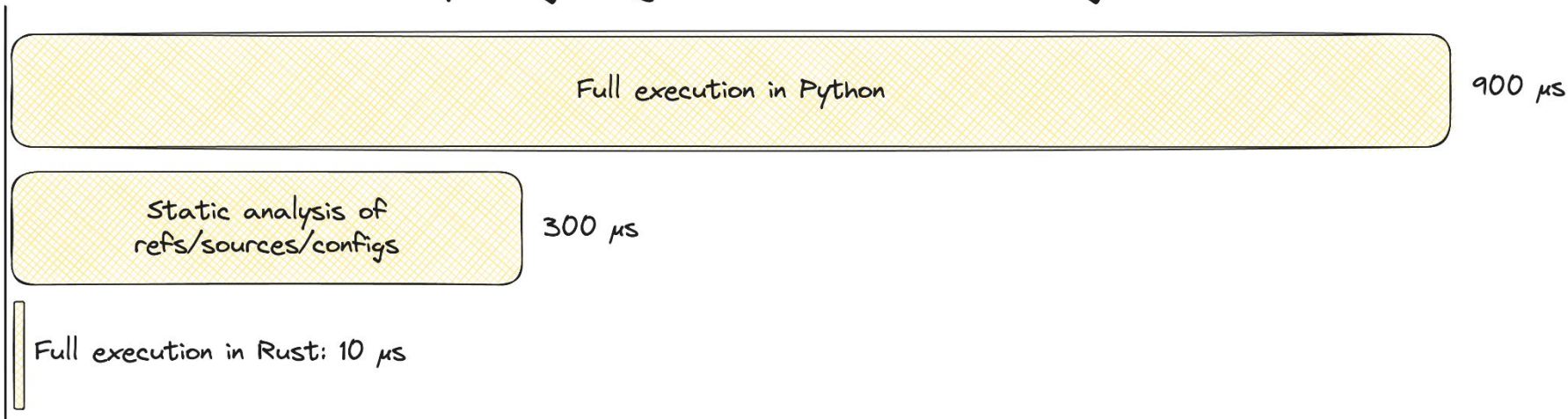
PARSING

Comparing Jinja evaluation strategies



PARSING WITH RUST

Comparing Jinja evaluation strategies



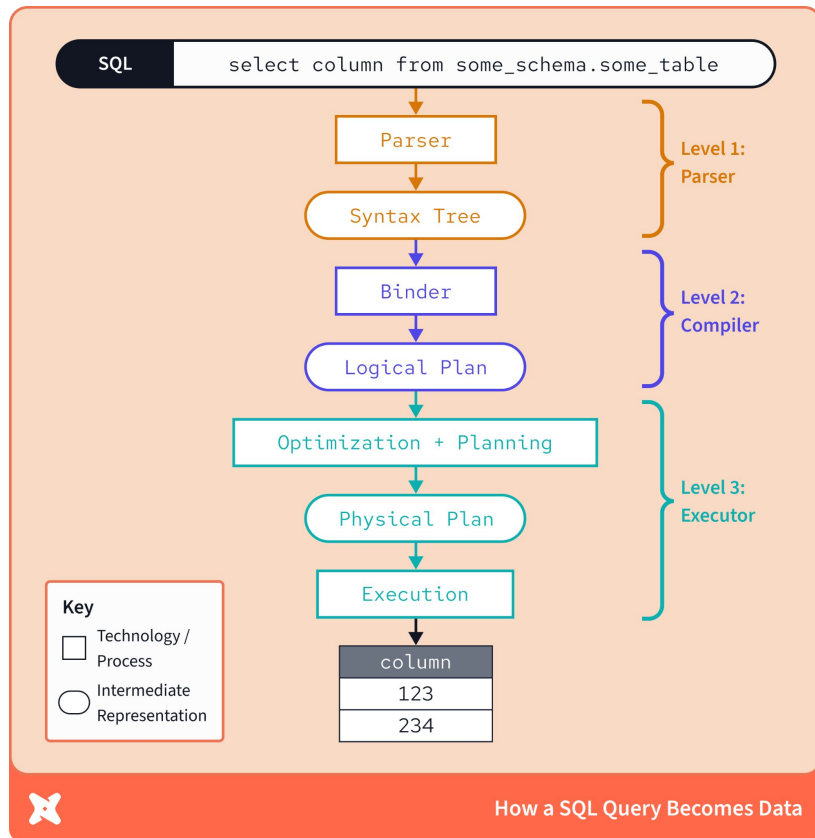
SQL COMPREHENSION

The Levels of SQL Comprehension: What can be validated	
Input	<code>select dateadd('day', 1, '2025-01-01') as new_day</code>
Level 1: Parser Syntax only	<code>select █████(█████, █, █████) as new_day</code>
Level 2: Compiler Level 1 + function names, argument count and data types	<code>select dateadd(datepart, int, date_or_time_expr) as new_day</code>
Level 3: Executor Level 2 + the data itself	<code>select dateadd('day', 1, '2025-01-01') as new_day</code> (validates everything!)

Source: <https://docs.getdbt.com/blog/faster-project-parsing-with-rust>

SQL COMPREHENSION

- **dbt Core:** let the Data Warehouse compile the SQL code
- **dbt Fusion:** let's compile and comprehend as much as possible without touching the DWH



dbt Fusion License

- dbt Core: **Apache 2**
- dbt Fusion: Mostly Elastic License v2 (**ELv2**)
 - You can use it for free, but you cannot provide it as a service for free
- Might include proprietary licensed enterprise features later
- For you most probably the license change won't make a difference

VISUAL STUDIO CODE EXTENSION

Let's go ahead and install the **VSCode Extension**, which will:

- Install dbt Fusion
 - Upgrade your dbt project to be 100% Fusion compatible
 - Compile your project for SQL Comprehension
-

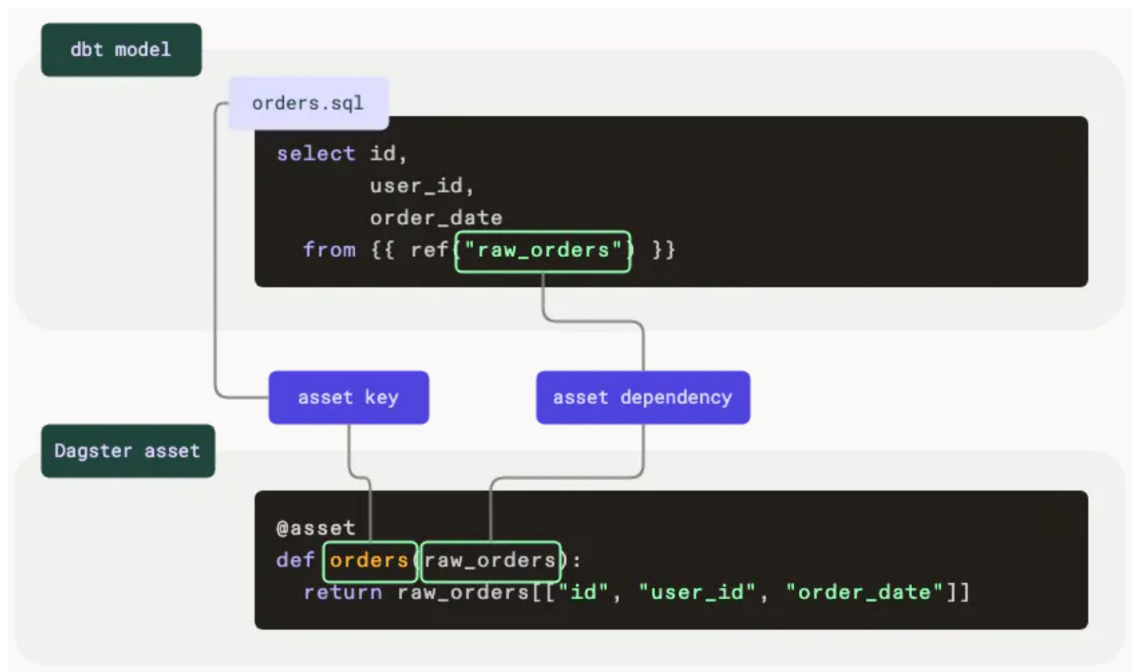
HERO

ORCHESTRATION

THE ORCHESTRATION LANDSCAPE



DAGSTER - SIMILAR DATA CONCEPTS



GREAT UI



☰

Overview

Runs

Assets

Deployment

materialize_dbt_models Job in dbt_dagster_project At 00:00 UTC Latest run: 15 Jan, 10:32 View 8 assets

Overview

Runs

Filter

Type an asset subset... (ex: ++dim_listings_w_hosts)

Clear query

Materialize selected

listings

scd_raw_listings

dbt snapshot scd_raw_listings

Materialized

15 Jan, 10:33

dbt

dim_listings_cleaned

Cleaned table which contains Airbnb...

Materialized

15 Jan, 10:33

dbt

hosts

scd_raw_hosts

dbt snapshot scd_raw_hosts

Materialized

15 Jan, 10:32

dbt

dim_hosts_cleaned

Cleaned table for the Airbnb hosts

Materialized

15 Jan, 10:32

dbt

reviews

fact_reviews

dbt model fact_reviews

Materialized

15 Jan, 10:32

dbt

seed_full_moon_dates

dbt seed seed_full_moon_dates

Materialized

15 Jan, 10:32

dbt

dim_listings_w_hosts

dbt model dim_listings_w_hosts

Materialized

15 Jan, 10:33

dbt

seed_full_moon_reviews

dbt model seed_full_moon_reviews

Materialized

15 Jan, 10:33

dbt

unique_id

materialize_dbt_models_w_10313

invocation_id

6d8b8365-92f4-4a93-8c40-c527d3d747df

Execution Duration

2.371628

Materialization system tags

Metadata plots

Execution Duration

Value

4.0

3.0

2.0

10:32:02 a.m.

10:32:14 a.m.

10:32:26 a.m.

10:32:38 a.m.

10:33:02 a.m.

10:33:14 a.m.

10:33:26 a.m.

10:33:38 a.m.

Timestamp

Code Version

efa504b664e0826bf27e5cbfabdc1ca443f69da1

Config

{}

Required resources

dbt

Metadata

table_schema

[Show Table Schema]

dagster_dbt/manifest

[DbtManifestWrapper]

(unserializable)

dagster_dbt/dagster_dbt_translator

[DagsterDbtTranslator]

View as Asset Graph

EASY TO DEBUG



Overview Runs Assets Deployment

Search... /

b5aef1ea View job View tags and config

Success Run of materialize_dbt_models @ b6dde2d1 9 assets 15 Jan, 10:32:29 0:01:13

Hide not started steps Re-execute all (*)

30s 60s 90s

Preparing (0) No steps are waiting to execute

Executing (0) No steps are executing

Errored (0) No steps have errored

Type a step subset (ex: dbtlea) Hide unselected steps

Events stdout stderr Filter... Levels (5)

TIMESTAMP	OP	EVENT TYPE	INFO
			step_keys ["dbtlearn_dbt_assets"] captured_logs View stdout / stderr
10:32:32.515	dbtlearn_dbt_assets	STEP_START	Started execution of step "dbtlearn_dbt_assets".
10:32:32.551	dbtlearn_dbt_assets	INFO	A dbt subsetted execution is not being performed. Using the default dbt selection arguments ["--select", "fq:.*"].
10:32:32.553	dbtlearn_dbt_assets	INFO	Copying "/Users/zoltantcoth/src/dbt-zero-to-hero/orchestration/dbtlearn/target/partial_parse.msgpack" to "/Users/zoltantcoth/src/dbt-zero-to-hero/orchestration/dbtlearn/target/dbtlearn_dbt_assets-b5aef1e-5c7bfdf/partial_parse.msgpack" to take advantage of partial parsing.
10:32:32.555	dbtlearn_dbt_assets	INFO	Running dbt command: "dbt build --select fq:.*".
10:32:43.377	dbtlearn_dbt_assets	STEP_OUTPUT	Yielded output "seed_full_moon_dates" of type "Nothing". (Type check passed). unique_id seed.dbtlearn.seed_full_moon_dates invocation_id 6d8b8365-92f4-4a93-8c40-c527d3d747df Execution Duration 3.139401

Capstone Project

Let's implement a **real-world pipeline end to end on your own**

- Clone the repo
- Get to know the data
- Read the instructions and solve the assignment
- My solution: [Solutions link](#)

If you started the course before 14 Mar 2025, go to dbtsetup.nordquant.com and go follow the instructions at the “Capstone-only Setup” tab before you start.

CONGRATS !

**FROM THIS POINT
THE SLIDES ARE
WORK IN PROGRESS**